

LAB 11

MIPS Assembly I: Machine, Arithmetic & Memory

The toolkit behind Labs 12, 13 and 14

Fetch-execute · registers · syscalls · arithmetic · HI/LO · memory

90-minute hands-on lab · B.Tech CS · Year 3

Anuj Kumar · Newton School of Technology

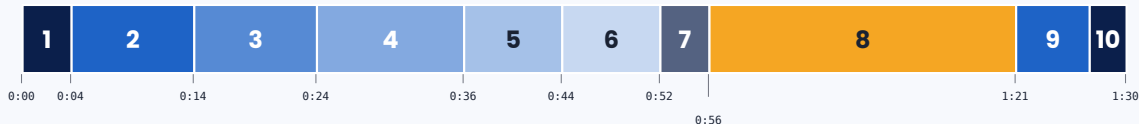
add \$t2,\$t0,\$t1

000000 01000 01001 01010 00000 100000

0x01095020

one instruction = one 32-bit word

Run of show: 90 minutes, typing along from minute 24



#	Segment	Min	What happens
1	Hook + roadmap	4	MIPS from Stanford (1984) to RISC-V (2021); how Labs 11-14 fit together
2	Machine model	10	CPU, memory, load/store, fetch-decode-execute, registers
3	Sheets and formats	10	The two reference sheets; R/I/J formats; encode and decode by hand
4	Demo 1	12	First program, syscalls (print, read, exit), pseudo-instructions
5	Demo 2	8	Arithmetic, HI/LO for mult/div, logic and shifts
6	Demo 3	8	Scalar memory: <code>la</code> vs <code>lw</code> , <code>sw</code> , base + offset, alignment
7	Bug hunt	4	Five broken snippets, pairs
8	Hands-on	25	Four practice problems + three finishers; you circulate
9	Debrief	6	Reference solutions, cold calls
10	Exit	3	Exit ticket; preview of Lab 12

By minute 90 you can...

1. Explain **fetch, decode, execute** and say what the PC does after every instruction.
2. Name the registers by convention and use only \$t, \$a, \$v, \$zero today.
3. Write a .data/.text program that reads integers, computes, prints and exits via `syscall`.
4. Use `add/sub/addi, mult/div` with **HI/LO**, logic and shifts; know which instructions are pseudo.
5. Tell `la` (an *address*) from `lw` (a *value*); reach the next word with `4($t0)`.
6. Encode and decode one R-type and one I-type instruction by hand.

Open-book lab.

Both reference sheets are open all 90 minutes. Memorise nothing. Understand everything.

Practice problems today

- Hello, MIPS!
 - Add Two Inputs
 - Quotient and Remainder
 - Swap Two Memory Variables
- Finishers:* Celsius to Fahrenheit, Multiply by 10 with shifts, $(a + b)(a - b)$

The four-lab arc: today is the toolkit for everything after

LAB 11 • today

MIPS Assembly I

Machine model, registers, syscalls, arithmetic, HI/LO, logic, scalar memory.

Practice

Hello, MIPS! • Add Two Inputs • Quotient and Remainder • Swap Two Memory Variables

Unlocks

```
li la move add sub  
addi mult div mfhi  
mflo and or sll srl lw  
sw syscall
```

LAB 12

Memory and arrays

Base register, offsets, pointer stride 4, first counted loop.

Portal

Access First Element of an Array • Sum of Array Elements • Maximum Element in an Array

Unlocks

```
lw 4($t0), sll indexing,  
slt, ble/bge, j
```

LAB 13

Branching, loops and conditional logic

Every control-flow shape, built on beq/bne/slt.

Portal

A Number Is Even or Odd • Find the Sum from 0 to 9 • Iterative Factorial of 5 • Iterative Factorial of Any Number

Unlocks

```
beq bne blt bgt, andi  
parity, mult/mflo loops
```

LAB 14

Functions, stack frames and recursion

Calling convention, \$ra, \$sp, saved registers.

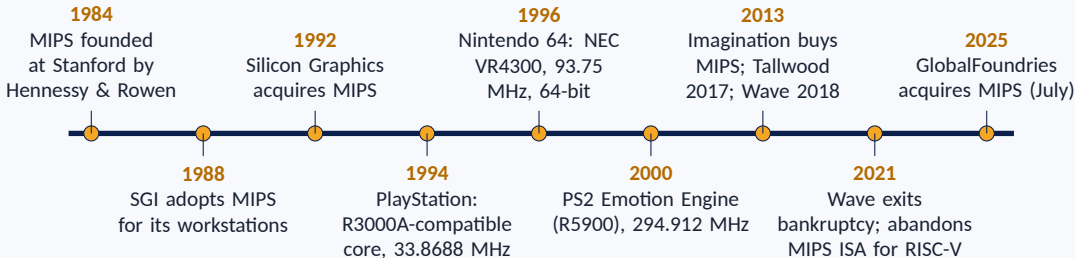
Portal

Recursive Factorial • Recursive Fibonacci

Unlocks

```
jal jr, $ra, $sp,  
$s0--$s7, push/pop  
frames
```

MIPS: 41 years from a Stanford lab to RISC-V



Why we start here. Fixed 32-bit instructions, three formats, 32 registers, load/store: the cleanest teaching ISA ever shipped. Hennessy and Patterson shared the 2017 Turing Award for this philosophy; RISC-V descends from it and we use both.

Chip War lens. An instruction set is a strategic asset, not just a technical artefact. ARM won mobile; MIPS lost it, sold itself four times, then abandoned its own ISA in 2021.

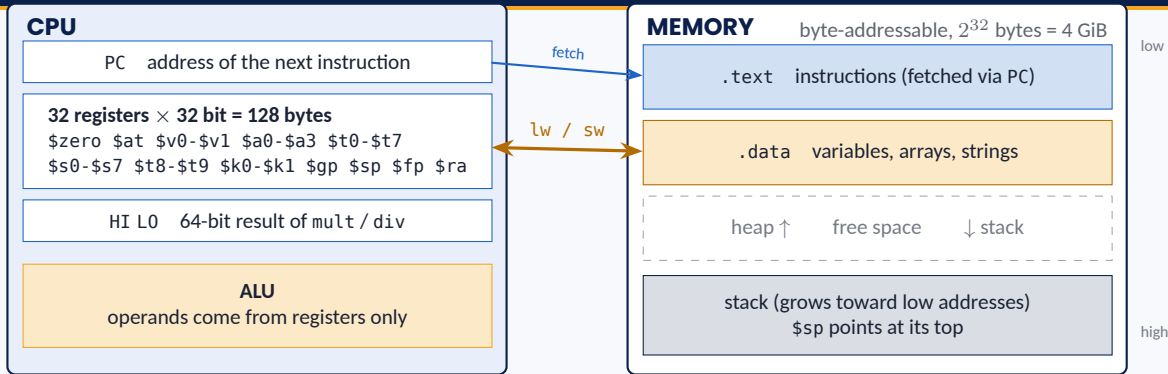
Where MIPS shipped: real chips, real clock speeds

Product	Year	CPU	Clock	Fact
SGI workstations	1988	MIPS R-series	—	<i>Jurassic Park, T2, The Abyss</i>
MIPS R3000/R3000A	1988+	R3000A	up to 40 MHz	About 32 MIPS
Sony PlayStation	1994	R3000A-compatible (LSI)	33.8688 MHz	No FPU, no MMU; 32-bit
Nintendo 64	Jun 1996	NEC VR4300 (R4300i family)	93.75 MHz	64-bit; about 125 MIPS
Sony PlayStation 2	2000	Emotion Engine R5900 (Sony, Toshiba)	294.912 MHz	2 × 16 MB RDRAM
Embedded	1990s+	Routers, printers, automotive	—	Licensable core

Lab sanity check. At ideal CPI = 1 one instruction takes 29.5 ns on the PlayStation CPU (33.8688 MHz). Program A today, about 12 real instructions, runs in roughly 0.35 μ s.

From this room. Every app on every phone is, at the bottom, a stream of 32-bit words like the ones we decode today.

The machine: two boxes and a narrow road between them



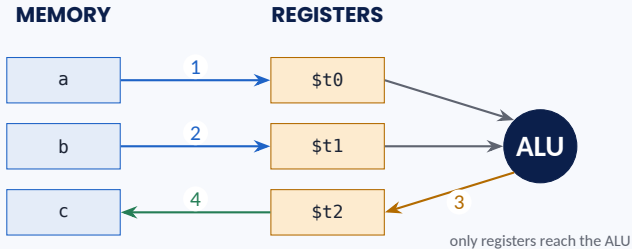
Memory is byte-addressable: a word is 4 bytes, so consecutive words are 4 apart. Every instruction is exactly 32 bits. The ALU sees registers only: to use memory you **load**, compute, **store**.

Load/store architecture: $c = a + b$ costs four instructions

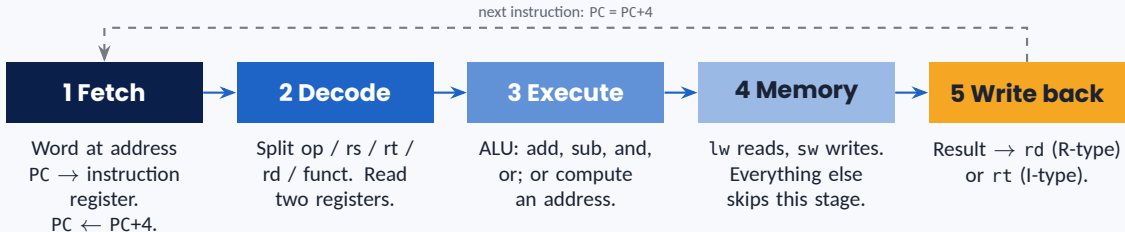
```
lw    $t0, a      # 1 memory -> reg
lw    $t1, b      # 2 memory -> reg
add   $t2, $t0, $t1 # 3 reg + reg
sw    $t2, c      # 4 reg -> memory
```

Why the restriction? Every non-memory instruction is *register-to-register*: fixed 32-bit format, fast decode, simple pipeline. x86 lets ALU instructions read memory; MIPS does not.

add \$t0, var, \$t1 is illegal: operands of add are registers.



Fetch, decode, execute: the loop the CPU never stops



Instruction	Fetch	Decode	Execute	Memory	Write back	Note
add \$t2,\$t0,\$t1	•	•	•		•	no memory access
lw \$t0,4(\$t1)	•	•	•	•	•	the only instruction that uses all five
sw \$t0,4(\$t1)	•	•	•	•		nothing to write back
beq \$t0,\$t1,L	•	•	•			may replace PC (Lab 13)

These five stages are the datapath you build in the single-cycle module, and the five pipeline stages afterwards. Fixed 32-bit instructions make stage 1 trivial: the next instruction is always at $PC + 4$.

32 registers: names, numbers, and who may destroy what

\$zero 0	\$at 1	\$v0 2	\$v1 3	\$a0 4	\$a1 5	\$a2 6	\$a3 7
\$t0 8	\$t1 9	\$t2 10	\$t3 11	\$t4 12	\$t5 13	\$t6 14	\$t7 15
\$s0 16	\$s1 17	\$s2 18	\$s3 19	\$s4 20	\$s5 21	\$s6 22	\$s7 23
\$t8 24	\$t9 25	\$k0 26	\$k1 27	\$gp 28	\$sp 29	\$fp 30	\$ra 31

\$v0–\$v1, \$a0–\$a3

Return values and arguments. **\$v0 = syscall service number, \$a0 = syscall argument.**

\$t0–\$t9

Temporaries: a callee may destroy them freely. **Today we use only these.**

\$s0–\$s7

Saved: a callee must preserve them (Lab 14).

\$sp \$fp \$ra

Stack pointer (grows *down*), frame pointer, return address written by `jal`.

\$zero, \$at

`$zero` is always 0 (writes ignored). `$at` is the assembler's scratch: **never use it.**

Two reference sheets, two jobs

	Sheet J: JHU CS333 Quick Reference	Sheet K: KTH Reference Sheet v1.12
Strength	Conventions, syscall table, worked examples	Exact semantics, opcode/funct, bit formats
Multiply	mult → HI:LO (64-bit), read with mflo/mfhi	mul rd, rs, rt, 32-bit result (op 28, funct 2)
Pseudo-ops	Lists blt ble bgt bge, li, la, move	Shows real expansions of li, la, ble, move, nop
Syscalls	Full table, services 1–10	None
Directives	.data .text .word .byte .space .ascii	Adds .align 2, .global, .ascii
Entry label	Uses both main: and __start: in different examples	Not specified

When they disagree: Sheet K is the hardware truth, Sheet J is the assembler's convenience, and **the portal is the final judge.**

Both sheets contain small errors (see the errata slide at the end). *Trust, but verify against the simulator.*

Every instruction is exactly 32 bits: three formats

R-type register arithmetic: add sub and or nor slt sll srl sra jr



I-type one register + 16-bit constant: addi andi ori lw sw lb sb beq bne lui slti



J-type 26-bit word address: j jal



Fixed width: fetch 4 bytes, decode while reading registers. x86 cannot.

rs, rt = the two read ports; **rd** = the write port: exactly the Module 7 register file.

I-type has no rd: the destination is rt (loads, addi, lui).

Be the assembler: encode `add $t2, $t0, $t1`

op	rs	rt	rd	shamt	funct
000000	01000	01001	01010	00000	100000
0 = R-type	\$t0 = 8	\$t1 = 9	\$t2 = 10	0	32 = add

0000 0001 0000 1001 0101 0000 0010 0000

0x01095020

Source line	Machine word
<code>li \$t0, 7</code> → <code>addiu \$t0,\$zero,7</code>	0x24080007
<code>li \$t1, 5</code> → <code>addiu \$t1,\$zero,5</code>	0x24090005
<code>add \$t2, \$t0, \$t1</code>	0x01095020

Some assemblers emit `ori $t0,$zero,7` (0x34080007) for `li`.

The CPU never sees `$t2` or “add”. It sees bits. **The program is a list of 32-bit numbers in memory.**

Reverse it: decode 0x8D280004

1000 1101 0010 1000 0000 0000 0000 0100

op	rs	rt	immediate
100011	01001	01000	000000000000000100
35 = lw	\$t1 = base	\$t0 = dest	4 = byte offset

lw \$t0, 4(\$t1)

Which register receives the data? rt. I-type has no rd, so for loads the destination sits in the rt slot.

Your turn (30 s): encode addi \$t2, \$t3, 5.
 Answer: op 001000, rs 01011, rt 01010, imm 5 →
 0x216A0005

Your program is numbers in memory; PC walks through them

Address	Word	Assembly	Fields	PC after
0x00400000	0x24080007	addiu \$t0,\$zero,7	op 9, rs 0, rt 8, imm 7 (li \$t0,7)	0x00400004
0x00400004	0x24090005	addiu \$t1,\$zero,5	op 9, rs 0, rt 9, imm 5 (li \$t1,5)	0x00400008
0x00400008	0x01095020	add \$t2,\$t0,\$t1	op 0, rs 8, rt 9, rd 10, funct 32	0x0040000C

Addresses step by 4 because every instruction is 32 bits. The address 0x00400000 is the usual start of the text segment in teaching simulators: read it off your portal's text view.

Program and data share one memory.

Instructions live in `.text`, variables in `.data`. The CPU cannot tell them apart by looking at bits: only PC versus `lw/sw` decides which is which. (Stored-program idea, von Neumann.)

Why this matters for Labs 12–14: loops (Lab 13) and function calls (Lab 14) are nothing more than instructions that put a different value into PC.

Three details every student gets wrong once

1. Sign-extend vs zero-extend the 16-bit immediate

Sign-extended `addi addiu lw sw lb slti`
and branch offsets

Zero-extended `andi ori xori` (logic works on
bit patterns)

`addi $t0,$t0,-1: imm 0xFFFF → 0xFFFFFFFF.`
`andi $t0,$t1,0xFFFF: mask 0x0000FFFF.`

2. `lb` vs `lbu` on the byte `0xF0`

`lb` → `0xFFFFFFFF0` (sign-extend)

`lbu` → `0x000000F0` (zero-extend)

3. “Unsigned” is a misnomer (Sheet K clarification)

`addu` / `addiu` do the *same arithmetic* as `add` / `addi`. The only difference: they do **not trap on overflow**.

`$zero` buys free instructions

`move $t0,$t1` = add zero to `$t1`

`li $t0,7` = `addiu $t0,$zero,7`

`nop` = `sll $zero,$zero,0` (shift zero into zero)

`b L` = `beq $zero,$zero,L`

Anatomy of a program: two sections, labels, directives

```

        .data                # 1 variables in memory
msg:    .asciiz "Sum = "    # 2 null-terminated str
arr:    .word 5, 13, -7     # 3 initialised words
buf:    .space 40           # 4 reserve 40 bytes
        .text              # 5 instructions here
        .globl main        # 6 export main
main:   li $v0, 10          # 7 entry point
        syscall            # 8 exit

```

Labels *always* end with a colon. Forget the exit syscall and the CPU keeps fetching the next bytes in memory as instructions.

Directive	Meaning
<code>.data</code> / <code>.text</code>	Data segment / code segment
<code>.asciiz</code>	String plus terminating zero byte
<code>.ascii</code>	String without terminator (Sheet K)
<code>.word</code>	4-byte value(s)
<code>.byte</code>	1-byte value(s)
<code>.space X</code>	Reserve X bytes
<code>.align 2</code>	Align to a multiple of 4
<code>.globl</code> / <code>.global</code>	Export a label

Program A: add two numbers and print the result

```

.data
msg:   .asciiz "Sum = "
       .text
       .globl main

main:
    li    $t0, 7           # a = 7
    li    $t1, 5           # b = 5
    add   $t2, $t0, $t1    # sum = a + b
    li    $v0, 4           # 4: print string
    la    $a0, msg         # $a0 = &msg
    syscall
    li    $v0, 1           # 1: print int
    move  $a0, $t2         # $a0 = sum
    syscall
    li    $v0, 10          # 10: exit
    syscall

```

Output: Sum = 12

- `li` loads a constant. Pseudo-instruction: one real instruction for small values.
- `add rd, rs, rt:` **destination first**, then the two sources.
- `la` loads the *address* of `msg`, not the characters.
- `syscall` is not an ALU op: it traps to the OS layer, which reads `$v0` to pick the service.

Predict, then run: `sub` gives 2; `mult+mflo` gives 35; `add \n to msg`.

syscall: your only window to the outside world (Sheet J)

\$v0	Service	Arguments	Result
1	print integer	\$a0 = value	—
4	print string	\$a0 = address	—
5	read integer	---	\$v0
8	read string	\$a0 buf, \$a1 n+1	—
9	allocate memory	\$a0 = bytes	\$v0 = addr
10	exit	---	—

Three-beat pattern:

1. `li $v0, service` 2. `move $a0, value` 3. `syscall`

```
# read an int, print it back
li    $v0, 5
syscall                # value -> $v0
move  $a0, $v0
li    $v0, 1
syscall
li    $v0, 10          # always exit
syscall
```

Classic bug: `li $a0, 42` then `syscall`
with a stale `$v0`: the wrong service runs.

Pseudo-instructions: what you write vs what the CPU runs

You write	Assembler emits	Why
li \$t0, 7	addiu \$t0,\$zero,7	Constant fits in 16 bits: one instruction
li \$t0, 0xabcd1234	lui \$at,0xabcd ; ori \$t0,\$at,0x1234	Immediate is only 16 bits, so two instructions
la \$t0, label	lui \$at,hi ; ori \$t0,\$at,lo	32-bit address
lw \$t0, var	lui \$at,hi(var) ; lw \$t0,lo(var)(\$at)	\$at holds the upper half
move \$t0, \$t1	addu \$t0,\$zero,\$t1	Add zero
b L	beq \$zero,\$zero,L	Always-taken compare
blt \$t0,\$t1,L	slt \$at,\$t0,\$t1 ; bne \$at,\$zero,L	Real MIPS has only beq / bne
bge \$t0,\$t1,L	slt \$at,\$t0,\$t1 ; beq \$at,\$zero,L	not ($a < b$)
ble \$t0,\$t1,L	slt \$at,\$t1,\$t0 ; beq \$at,\$zero,L	Sheet K's own example
nop	sll \$zero,\$zero,0	Shift zero into zero

Consequence: count *real* instructions: la costs 2, bge costs 2. The branch rows preview Labs 12 and 13.

Arithmetic: registers are your variables

Instruction	Effect	Note
add rd,rs,rt	rd = rs + rt	traps on signed overflow
addu rd,rs,rt	rd = rs + rt	wraps silently
addi rt,rs,imm	rt = rs + signext(imm)	no subi: use -imm
sub rd,rs,rt	rd = rs - rt	traps on overflow
subu rd,rs,rt	rd = rs - rt	wraps silently
mul rd,rs,rt	rd = low 32 bits	Sheet K; HI lost
move rd,rs	rd = rs	pseudo: addu rd,\$zero,rs

Overflow, concretely: $2^{31} - 1 = 2,147,483,647 = 0x7FFFFFFF$. add with 1 raises an exception; addu returns $0x80000000 = -2,147,483,648$.

$x = (a + b) - c$ $a=7, b=3, c=4$

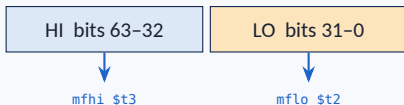
```
li    $t0, 7      # a
li    $t1, 3      # b
li    $t2, 4      # c
add   $t3, $t0, $t1 # t = a+b
sub   $t3, $t3, $t2 # t = t-c
# $t3 = 6
```

Variable	Register
a, b, c	\$t0, \$t1, \$t2
temp / result x	\$t3 (reused as source and destination)

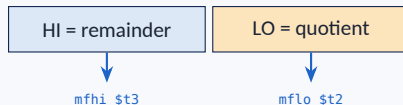
Destination comes *first*. A register may be both source and destination: `sub $t3,$t3,$t2` is legal and common.

mult and div: results land in HI and LO

mult \$t0, \$t1 ($32 \times 32 \rightarrow 64$ bits)



div \$t0, \$t1 ($\$t0 / \$t1$)



Operation	HI	LO	Reading
100000×100000	0x00000002	0x540BE400	product = 10,000,000,000 = $2 \cdot 2^{32} + 1,410,065,408$
$12! = 479,001,600$	0x00000000	0x1C8CFC00	fits in 32 bits: HI is 0
$13! = 12! \times 13$	0x00000001	0x7328CC00	true value 6,227,020,800; <i>LO alone reads 1,932,053,504</i>
$17 \div 5$	2	3	quotient in LO, remainder in HI
$-17 \div 5$	-2	-3	truncates toward zero; remainder takes the dividend's sign (verify on the portal)

Lab 13 preview: Iterative Factorial of Any Number overflows at 13!. HI tells you when. `mul rd, rs, rt` keeps only LO, so it cannot warn you.

`div` by zero gives an undefined result on MIPS (no trap). Guarding it needs a branch: Lab 13.

Logic and shifts: bits in, bits out

8-bit view: $a = 1100\ 1010$ (0xCA), $b = 0000\ 1111$ (0x0F)

Instruction	Result	Hex
<code>and \$t2,\$t0,\$t1</code>	0000 1010	0x0A
<code>or \$t2,\$t0,\$t1</code>	1100 1111	0xCF
<code>xor \$t2,\$t0,\$t1</code>	1100 0101	0xC5
<code>nor \$t2,\$t0,\$t1</code>	0011 0000	0x30

32-bit view:

Instruction	Result	Meaning
<code>sll \$t1,\$t0,3</code>	0x00000078	$0x0F \times 8 = 120$
<code>srl \$t1,\$t0,4</code>	0x0FFFFFFF	$\$t0 = -16 = 0xFFFFFFF0$; zero fill
<code>sra \$t1,\$t0,4</code>	0xFFFFFFFF	sign fill: $-16/16 = -1$

Idioms you will reuse

`andi $t1,$t0,1` → parity test (*Lab 13: even/odd*)

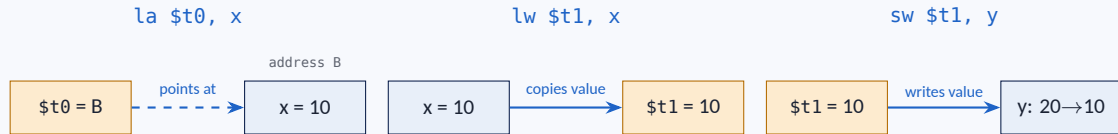
`sll $t1,$t0,2` → $\times 4$, index to byte offset (*Lab 12: arrays*)

`nor $t1,$t0,$zero` → bitwise NOT

`srl $t1,$t0,26` → extract the opcode field

Decode in code: $0x8D280004 \gg 26 = 35 = \text{lw}$. Shifts and masks pull fields out of an instruction word. `srl` zero-fills, `sra` sign-fills: the classic mix-up.

Scalar memory: `la` gives the address, `lw` the value, `sw` writes



```

.data
x:   .word 10
y:   .word 20
.text
la   $t0, x      # $t0 = &x (address)
lw   $t1, 0($t0) # $t1 = 10 (value)
lw   $t1, x      # same, via label

```

The classic bug: `la $a0, x` then `print-integer` prints the *address* (a large number), not 10. Use `lw $a0, x`.

`lw $t1, x` is a pseudo-instruction: `lui $at, hi(x)` then `lw $t1, lo(x)($at)`.

Program D: update a variable, reach the next word

```

.data
x:    .word 10
y:    .word 20
.text
.globl main
main:  lw    $t0, x           # $t0 = 10
      addi  $t0, $t0, 5      # $t0 = 15
      sw    $t0, x           # x = 15
      la    $t1, x           # $t1 = &x
      lw    $t2, 4($t1)      # $t2 = y = 20
      add   $t2, $t2, $t0    # 35
      li    $v0, 1
      move  $a0, $t2
      syscall                # prints 35
      li    $v0, 10
      syscall

```

Output: 35

Address	Before	After
B (x)	10	15
B+4 (y)	20	20

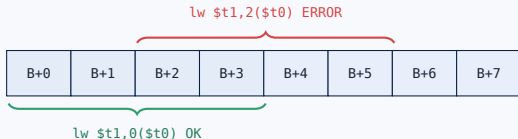
Effective address = base register +
signext(offset)

lw \$t2, 4(\$t1) reads address B+4: the
word right after x.

lw \$t2, -4(\$t1) would read the word
before. Negative offsets are fine.

Lab 12 in one sentence: two adjacent words
are the smallest array. Give the base register
a longer list and step it by 4.

Alignment and endianness: two ways memory surprises you



A word access needs an address that is a multiple of 4, or the CPU raises an **address error exception**. `lb/sb` have no such restriction. After a `.byte` declaration put `.align 2` before the next `.word`.

Word 0x12345678 stored at B

	B+0	B+1	B+2	B+3
big-endian	12	34	56	78
little-endian	78	56	34	12

`lb $t1, 0($t0)` gives 0x12 (big) or 0x78 (little).

MIPS hardware is bi-endian; the *portal simulator* picks one. Network byte order is big-endian, x86 is little-endian.

Bug hunt: one primary bug per snippet (pairs, 3 minutes)

S1 prints nothing / wrong thing

```
li    $a0, 42
syscall
```

S2 prints a huge number, not 42

```
x:    .word 42
...
la    $a0, x
li    $v0, 1
syscall
```

S3 prints 0, want 42

```
li    $t0, 6
li    $t1, 7
mult  $t0, $t1
li    $v0, 1
move  $a0, $t2
syscall
```

S4 quotient and remainder swapped

```
div   $t0, $t1    # 17 / 5
mfhi  $s0          # "quotient"
mflo  $s1          # "remainder"
```

S5 address error

```
la    $t0, x
lw    $t1, 2($t0)
```

Rules: find the bug, state the symptom, fix in one line.
Answers on the next slide.

Bug hunt: answers

	Root cause	Fix
S1	\$v0 never set. syscall runs whatever service number was left there.	li \$v0, 1 before syscall
S2	la loads the <i>address</i> of x; print-integer prints that address.	lw \$a0, x
S3	mult writes HI and LO, not a general register. \$t2 was never assigned.	mflo \$a0 after mult
S4	Roles reversed: LO holds the quotient (3), HI the remainder (2).	mflo \$s0; mfhi \$s1
S5	Word load at an address that is not a multiple of 4.	Offsets 0, 4, 8...; lb for bytes

Debugging order used all semester: (1) read the symptom, (2) write the trace table, (3) find the first row that differs from what you expect, (4) fix *that* line only.

Hands-on: 25 minutes, four practice problems

#	Problem	Tools	Min	Test cases
1	Hello, MIPS!	.asciiz, la, syscall 4	4	prints Hello, MIPS!
2	Add Two Inputs	syscall 5 twice, move, add	7	(3,4) → 7; (-5,12) → 7
3	Quotient and Remainder	div, mflo, mfhi	8	(17,5) → 3, 2; (100,7) → 14, 2
4	Swap Two Memory Variables	lw, sw, labels	6	x=10, y=20 → prints 20 then 10
★	Celsius to Fahrenheit	mult, div, addi	—	100 → 212; 37 → 98; -40 → -40
★	Multiply by 10 with shifts	sll, add	—	7 → 70; -3 → -30
★	$(a + b)(a - b)$	registers as variables	—	(7,3) → 40; (2,9) → -77

Before you raise a hand: have a trace table on paper. I will ask to see it.

Checkpoints

1:05 problems 1–2 submitted
 1:15 problem 3 submitted
 1:19 two minutes left

Code craft (10 pts): comments
 3, register discipline 2, exit 2,
 sensible labels 2, no \$at 1

Match the portal's starter code and output text *exactly*: one stray space or newline fails the auto-grader.

Plans, no code: read the sheet, then write

2. Add Two Inputs

1. `syscall 5` → value in `$v0`
2. Copy it to `$t0`
3. Read again → `$t1`
4. `add` into `$t2`
5. Print integer, exit

Watch: the second read overwrites `$v0`. Copy the first value out *before* reading again.

3. Quotient and Remainder

1. Read `a`, then `b`
2. `div $t0,$t1`
3. `mflo` = quotient
4. `mfhi` = remainder
5. Print each; newline between

Watch: LO is the quotient. Try `-17` and `5`, then predict before you run. `b = 0` is undefined: Lab 13 guards it.

4. Swap Two Memory Variables

1. `lw x` into `$t0`
2. `lw y` into `$t1`
3. `sw $t1` into `x`
4. `sw $t0` into `y`
5. Reload and print `x`, then `y`

Watch: if you `sw` before loading *both*, one value is gone. Registers are the temporary.

Sheet J: `syscall` numbers, `div/mflo/mfhi`. **Sheet K:** exact `lw/sw` and shift semantics. Finishers: multiply first, then divide; shifts replace multiplication; two temporaries, then one `mult`.

Reference solutions: problems 1 and 2

1. Hello, MIPS!

```

        .data
msg:    .asciiz "Hello, MIPS!\n"
        .text
        .globl main
main:   li    $v0, 4
        la    $a0, msg
        syscall
        li    $v0, 10
        syscall

```

The `z` in `.asciiz` adds the terminating zero byte.
Without it, `print-string` runs past the end of the text.

2. Add Two Inputs

```

        .text
        .globl main
main:   li    $v0, 5
        syscall
        move  $t0, $v0      # first value out of $v0
        li    $v0, 5
        syscall
        move  $t1, $v0
        add   $t2, $t0, $t1
        li    $v0, 1
        move  $a0, $t2
        syscall
        li    $v0, 10
        syscall

```

Trace: input 3, 4 → `$t0=3, $t1=4, $t2=7`. Match the portal's exact output text.

Reference solution: 3. Quotient and Remainder

```

.data
nl:    .ascii "\n"
.text
.globl main
main:  li    $v0, 5
      syscall
      move  $t0, $v0    # a
      li    $v0, 5
      syscall
      move  $t1, $v0    # b
      div   $t0, $t1    # LO = a/b, HI = a mod b
      mflo  $t2         # quotient
      mfhi  $t3         # remainder

```

```

li    $v0, 1
move  $a0, $t2
syscall           # print quotient
li    $v0, 4
la    $a0, nl
syscall           # newline
li    $v0, 1
move  $a0, $t3
syscall           # print remainder
li    $v0, 10
syscall

```

Trace 17, 5: $\$t0=17$, $\$t1=5$; after `div`: $LO=3$, $HI=2$; prints 3 newline 2. For $-17, 5$ the expected answer is -3 and -2 .

Ask a student: *why is the print sequence repeated?*
Because $\$v0$ and $\$a0$ are overwritten by every `syscall` setup. Lab 14 turns this pattern into a function.

Reference solution: 4. Swap Two Memory Variables

```

.data
x:    .word 10
y:    .word 20
nl:   .asciiz "\n"
.text
.globl main
main:  lw    $t0, x      # t0 = 10
      lw    $t1, y      # t1 = 20
      sw    $t1, x      # x = 20
      sw    $t0, y      # y = 10

```

```

lw    $a0, x
li    $v0, 1
syscall           # prints 20
la    $a0, nl
li    $v0, 4
syscall
lw    $a0, y
li    $v0, 1
syscall           # prints 10
li    $v0, 10
syscall

```

Step	\$t0	\$t1	x	y
start	—	—	10	20
lw x	10	—	10	20
lw y	10	20	10	20
sw x	10	20	20	20
sw y	10	20	20	10

Finishers (solutions in the script): $C \times 9 \div 5 + 32$ multiplies first so integer division loses nothing;
 $x \times 10 = (x \ll 3) + (x \ll 1)$.

Trust, but verify: errors hiding in the two sheets

#	Sheet	Issue	Correct
1	J	subu \$t1,\$t6,\$t7 comment says \$t1 = \$t6 + \$t7	Should be —
2	J	string1 .asciiz ...: label without a colon, though the sheet says labels always take one	string1: .asciiz ...
3	J	Load/store examples use __start: and a bare done; others use main:	Use main: and end with the exit syscall
4	J	addi \$t0, 1 two-operand shorthand; some assemblers reject it	addi \$t0,\$t0,1
5	K	move \$t0,\$t1 shown as addi \$t0,\$t1,\$zero: addi takes an immediate, not a register	addu \$t0,\$zero,\$t1
6	K	xori semantics typo rerg(rs)	reg(rs)
7	J vs K	Multiply: mult+mflo vs mul rd,rs,rt	Both legal; different encodings
8	J vs K	J lists blt ble bgt bge; K expands only ble	Others use the same slt trick

Every reference has bugs. The hardware is the only authority: when a sheet contradicts the simulator, believe the simulator, then find out *why*.

Cold calls: name first, then the question

1. Why can't you write `add $t0, var, $t1`?
2. What does `$zero` buy the ISA?
3. Which pipeline stages does `add` skip? `sw`?
4. Decode `0x012A4020`.
5. `add` vs `addu`: what really differs?
6. `la $t0,x` vs `lw $t0,x`: what is in `$t0`?
7. After `mult`, where is the result? After `div`?
8. `13!` in 32 bits: what does `LO` hold, and where do you see the overflow?
9. `srl` vs `sra` on `-16` by `4`?
10. Why is `sll $t1,$t0,2` useful for arrays?
11. What happens on `lw $t1,2($t0)`?
12. What does `PC` hold, and how does it change?
13. Why is `li $t0,0xabcd1234` two instructions?
14. Which R-type fields map to the register-file ports?

Then, 2 minutes: “What mistake did you make most often today? Tell your neighbour.” Share the top two with the room.

What each later portal problem borrows from today

Lab	Portal problem	Lab 11 tool it needs	New in that lab
12	Access First Element of an Array	la, lw 0(\$t0), print-int syscall	.word lists, array = base + offsets
12	Sum of Array Elements	add accumulator, addi by 4	first counted loop (bge, j)
12	Maximum Element in an Array	compare with slt, move	sll index scaling, conditional update
13	A Number Is Even or Odd	andi \$t1,\$t0,1, syscall 5, print-string	beq/bne, two-way branch
13	Find the Sum from 0 to 9	add, addi, counter	the loop skeleton, trace tables
13	Iterative Factorial of 5	mult + mflo	loop with a running product
13	Iterative Factorial of Any Number	syscall 5, mult/HI (overflow at 13!)	input checks, edge cases (0!, 1!)
14	Recursive Factorial	mult/mflo, register conventions	jal/jr, \$ra, \$sp, stack frame
14	Recursive Fibonacci	add, \$s vs \$t rules	two recursive calls, saving \$s registers

Readiness check before Lab 12: you can (1) print an integer and a string, (2) read an integer, (3) la + lw from a label, (4) explain why array steps are multiples of 4. If not, redo Program D.

Exit ticket, and next lab: memory and arrays

Exit ticket (2 min)

1. `lw $t0, 4($t1)` with `$t1 = 0x1000`: effective address?
2. After `div $t0,$t1`, which register instruction gives the remainder?
3. `la $t0, x` vs `lw $t0, x`: which puts 10 in `$t0`?

Lab 12 portal: Access First Element of an Array, Sum of Array Elements, Maximum Element in an Array.

Teaser: Lab 12 starts here

```
.data
arr:  .word 10, 20, 30, 40, 50
.text
la    $t0, arr
lw    $t1, 0($t0)    # arr[0]
lw    $t2, 4($t0)    # arr[1]
lw    $t3, 8($t0)    # arr[2]
```

Bring today's *Program D* and your trace tables. Lab 12 is Program D with more than two words.

LAB 11 • ONE-PAGE CHEAT SHEET

Copy this. Then trace before you run.

PRINT INT	li \$v0,1 ; move \$a0,\$reg ; syscall
PRINT STR	li \$v0,4 ; la \$a0,label ; syscall
READ INT	li \$v0,5 ; syscall ; value in \$v0 (copy it out!)
EXIT	li \$v0,10 ; syscall
MULTIPLY	mult \$a,\$b ; mflo \$r (low 32) ; mfhi \$h (high 32)
DIVIDE	div \$a,\$b ; mflo \$q (quotient) ; mfhi \$r (remainder)
SHIFT/MASK	sll \$r,\$x,k ($x \cdot 2^k$) ; andi \$r,\$x,1 (bit 0)
ADDRESS	la \$t0,x ($\rightarrow$ address)
VALUE	lw \$t1,x or lw \$t1,0(\$t0) ($\rightarrow$ value)
STORE	sw \$t1,x or sw \$t1,4(\$t0)

Registers \$v0 service • \$a0 argument • \$t temporaries • \$s preserved • \$sp grows down

Formats R: op rs rt rd shamt funct • l: op rs rt imm16 • J: op addr26

Rules word = 4 bytes • lw/sw address multiple of 4 • destination first • LO = quotient

Sources: MIPS Quick Reference (JHU CS333); MIPS Reference Sheet v1.12 (D. Broman, KTH); MIPS Technologies history; SGI, Sony, Nintendo and Toshiba product records. Next: Lab 12 (arrays), Lab 13 (control flow), Lab 14 (stack and recursion).